Bulkhead Pattern

Category: Resilience **One-liner:** Partition a system into isolated compartments with their own dedicated resources, so a failure or overload in one cannot consume the resources everything else needs.

Part 0 — Concepts you need first

What is a thread pool?

A fixed collection of worker threads a program keeps ready. When work arrives, a free thread picks it up; when the work finishes, the thread returns to the pool.

Pools are fixed-size because threads are expensive — each needs memory (typically ~1 MB of stack), and having thousands means the operating system spends more time switching between them than doing work.

The consequence that drives this entire chapter: the pool is a *shared, finite resource*. If something exhausts it, **every** piece of work in that program stops — not just the work that caused the problem.

What does "resource exhaustion" mean?

Using up all of a limited resource so nothing else can get any. The resources that matter here:

- **Threads** — all busy, so no new work can start
- **Memory** — allocated, so new allocations fail
- **Database connections** — connection pools are also fixed-size and also exhaustible
- **Network bandwidth / file handles / CPU**

What is a "noisy neighbour"?

A component that consumes disproportionate shared resources and degrades everything sharing them. The term comes from cloud hosting, where one tenant's heavy usage slows other tenants on the same physical machine. In your own application, one slow dependency can be a noisy neighbour to every other feature.

Why does one slow dependency block unrelated work?

This is the crux, so it's worth walking through concretely.

Your service has **one thread pool of 200 threads**. It calls two dependencies: Inventory (usually busy) and Shipping (usually quiet).

Inventory becomes slow — 30 seconds per call instead of 50 milliseconds.

1. Requests needing Inventory arrive and each takes a thread
2. Each thread blocks for 30 seconds waiting
3. At any moderate request rate, all 200 threads end up blocked on Inventory
4. A request arrives needing only Shipping — **there is no thread available to serve it**

Shipping is perfectly healthy. Its requests still fail. They fail because Inventory ate all the threads. That's the failure this pattern prevents.

Part 1 — Where the name comes from

A ship's hull is divided into **watertight compartments** called bulkheads. If the hull is breached, water floods **one compartment** rather than the whole ship. The ship stays afloat, buying valuable time for rescue.

Without bulkheads, a single breach floods everything and the ship sinks rapidly. The failure — one hull breach — propagates across the entire ship and causes total system failure. That's a **cascading failure in a physical system**.

In distributed systems, a bulkhead is a mechanism that **isolates different parts of the system so that if one part fails, it doesn't cause the rest to fail.**

Part 2 — The problem: failure propagation

In an ideal world services would work perfectly all the time. In reality they fail — bugs, resource exhaustion, network issues, and more. Each service has its own responsibilities, resources, and failure modes.

When a service fails, it causes a **ripple effect**: the failure of one service leads to the failure of others that depend on it.

The e-commerce walkthrough

A microservices-based e-commerce platform with separate services for user management, inventory management, payment processing, and so on.

1. The **inventory service** fails due to a database outage
2. The **user service**, which relies on inventory to display product availability, **also starts failing**
3. The **payment service**, which checks inventory before processing payments, **likewise fails**
4. In a short time, **the entire platform is unavailable**

All because of a failure in one service. This is a classic cascading failure.

Part 3 — The solution: isolation

The essence of the Bulkhead pattern is **isolation**. Like the watertight compartments in a ship, it partitions system components into isolated units. Each is designed to operate **independently of the others**, so if one fails, the others continue functioning.

Think of bulkheads as **failure containment**. By isolating parts of the system, you limit the **scope** of any potential failure. A failure in one part doesn't automatically mean failure of the whole — unaffected parts still provide some level of service, maintaining availability as much as possible.

Applied to the e-commerce example

If the inventory service were isolated into its own bulkhead, its failure wouldn't directly impact the user and payment services. They could continue operating, providing limited functionality — browsing products, viewing past orders — despite inventory being down.

The half people miss

There's a second, subtler benefit. By preventing the inventory failure from **consuming system-wide resources** like CPU or memory, the pattern ensures those resources **remain available for the remaining services**.

> Isolating *resources* matters as much as isolating *code*. Two services can be perfectly separate in the source tree and still kill each other if they draw from the same thread pool. The bulkhead is about the resources, not the modules.

Not a magic solution

The course is careful here, and it's worth internalising: bulkheads don't make failures disappear. If a ship has a significant hull breach it might eventually sink despite its bulkheads. Likewise, if a distributed system experiences a major failure like a total network outage, it may become unavailable despite bulkheads.

What they do is **buy time and maintain some level of service**, significantly mitigating the impact.

Part 4 — Architecture

The architecture revolves around two concepts: **isolation** and **resources**.

What is a bulkhead, concretely?

At a high level, a bulkhead is simply a **partition** within the system. It could be:

- A separate **service**
- A group of related **processes**
- A set of **threads** — a thread pool, the most common implementation
- A dedicated **CPU or memory area**

The key requirement is that each operates **independently** of the others.

What are the resources?

Anything the bulkheads need to function: CPU time, memory, network bandwidth, database connections. Each bulkhead gets a **dedicated share**, ensuring a failure in one doesn't consume all the system's resources.

Step 1 — Divide the system into bulkheads

How you divide depends on your system's specifics and which failures you want to isolate. Common approaches:

- Isolate services that are **prone to failure**
- Isolate services handling **particularly resource-intensive tasks**
- Group services by **function**, or by the **type of users they serve**

In the e-commerce example, you'd create separate bulkheads for user management, inventory management, payment processing, and so on. Each operates independently with its own resources.

The aim: if one service fails **or becomes a resource hog**, the others aren't directly affected.

Step 2 — Allocate resources

Allocation should be **proportional to the needs and importance** of each bulkhead. A resource-intensive or critical service gets more than a less demanding or less critical one.

In the e-commerce platform you might allocate more to **payment processing** (which handles financial transactions) than to **user management** (which mainly handles profiles). The specific allocation depends on:

- Expected load on each service

- Its performance requirements
- The consequences of its failure

This is the critical mechanism: if a bulkhead fails and starts consuming resources excessively, the allocation **guarantees other bulkheads still have resources available**.

Step 3 — Manage interactions between bulkheads

Since each bulkhead operates independently, interactions between them must be carefully managed or failures propagate anyway.

Keep interactions to a minimum. The more a bulkhead depends on other bulkheads, the higher the chance of a failure spreading.

When interactions are necessary, isolate them. If the user management service must interact with inventory, it could do so via a **message queue** or an **API gateway**. That way, if inventory fails, the failure doesn't directly impact user management — the interaction stays isolated and manageable.

> Why a message queue helps: with a direct call, the caller waits for a response and its thread is held while the callee struggles. With a queue, the caller drops a message and moves on immediately. The failure becomes "messages accumulate in a queue" instead of "caller's threads are all blocked."

Part 5 — Inner workings

Isolation in practice

Maintaining independence involves careful design and implementation choices. Each bulkhead might:

- Run in its **own process or container**
- Have its **own database and cache**
- Use its **own thread pool**

But the course makes a point that's easy to skip past and shouldn't be:

> Achieving isolation is **not just about technical measures**. It also requires an **organisational commitment** to maintaining independence. Changes to one bulkhead should not require changes to others. Interdependencies must be minimised and carefully managed.

In other words, you can build perfect technical isolation and then destroy it over six months by letting teams add convenient direct dependencies between compartments. The isolation has to be defended, not just built.

Resource allocation — fairness and efficiency

The aim is **fairness** (each bulkhead gets the resources it needs) and **efficiency** (resources aren't wasted). In practice this involves:

- Setting **resource quotas or limits** per bulkhead
- Scheduling resources fairly and efficiently
- **Dynamically adjusting** allocation based on real-time conditions

For instance, if one bulkhead is experiencing a surge while another is idle, the system might temporarily reallocate some resources from the idle one to the busy one, improving overall performance and responsiveness.

Failure detection and mitigation

The ultimate test is how the pattern handles failures. When one occurs within a bulkhead, the system needs to **detect it, isolate it, and mitigate its impact**. This typically involves monitoring, alerting, and automatic recovery:

- The system **continuously monitors** the health and performance of each bulkhead
- If a problem is detected, it **generates alerts** and starts **automatic recovery**
- If a bulkhead starts consuming resources excessively, the system might automatically **throttle its resource usage or restart it**
- If it remains unresponsive, the system might **reroute traffic** to other bulkheads or **bring up new instances** to handle load

It's not a silver bullet

The pattern can't prevent failures from happening and can't guarantee 100% uptime or performance. It's one tool in the toolbox, to be complemented with **redundancy, replication, load balancing, and caching**.

Part 6 — Worked example: thread-pool bulkheads in Java

The scenario

An **Order Service** responsible for creating orders. It must communicate with two others:

- **Inventory Service** — often gets high traffic
- **Shipping Service** — usually has less load

Without bulkheads: a spike in requests to Inventory could clog the Order Service, preventing it from completing its tasks involving Shipping.

Creating separate thread pools

Java's `ExecutorService` is well suited to creating thread pools:

```
ExecutorService inventoryExecutor = Executors.newFixedThreadPool(100);
ExecutorService shippingExecutor  = Executors.newFixedThreadPool(50);
```

Reading it: `newFixedThreadPool(100)` creates a pool with exactly 100 threads. Submit more than 100 tasks and the extras queue until a thread frees up.

The pool sizes should be determined by the load each service is expected to handle. Inventory gets the larger pool (100) because it typically faces more traffic; Shipping gets 50.

These two pools are the bulkheads. They are separate compartments; threads in one cannot be consumed by work destined for the other.

Using them

```
public void createOrder(Order order) {
    inventoryExecutor.submit(() -> {
        inventoryService.checkAndUpdateInventory(order);
    });

    shippingExecutor.submit(() -> {
        shippingService.scheduleShipping(order);
    });
}
```

`createOrder` initiates tasks that interact with both services. Each task is **submitted to its own executor**, ensuring a slowdown in one service doesn't affect the other.

The bulkhead in action

Now replay the failure scenario:

1. A significant number of requests start slowing down the **Inventory Service**
2. Threads in `inventoryExecutor` get stuck or slowed — potentially all 100 of them
3. **The 50 threads in `shippingExecutor` are completely unaffected**, because they're a separate pool
4. `createOrder` can **continue scheduling shipping** even while inventory checks are delayed

The system uses its resources efficiently and provides a faster response where it can.

Compare this to the single-pool version: all 200 threads blocked on Inventory, and shipping requests failing for no reason of their own. **The only change is drawing from two buckets instead of one.**

What the example deliberately leaves out

The course itself raises these as questions leading into the next section:

- **Error handling** — what if a service doesn't respond at all, or throws an exception?
- **Timeouts** — a stuck thread needs a bound, or it's stuck forever *(pair with Circuit Breaker)*
- **Pool sizing** — how do you actually determine those numbers?

Also worth noticing: `submit()` returns a `Future` that this code ignores, so **failures are silently swallowed**. Real code would capture and handle the result.

Part 7 — Performance implications

Benefits

Efficient resource utilization. Even when one service is sluggish or non-responsive, the system continues processing other tasks. The risk of the entire system freezing or significantly slowing due to one faulty component is mitigated.

Load balancing. Allocating different thread pools to various tasks or services distributes the system's load. The distribution can be fine-tuned based on the nature and expected load of each.

Response time. By ensuring a delay in one part doesn't affect others, overall response time improves. Even if some requests are delayed by an issue in one component, others are served swiftly.

Considerations

Determining bulkhead sizes. One of the real challenges. Ideally sizes are decided by expected load and the nature of the tasks — but striking the right balance often involves **a degree of trial and error**.

> A practical starting point: size each pool to the concurrency that dependency can actually handle. If Inventory can serve 100 concurrent requests before degrading, a pool larger than 100 just queues work inside the dependency instead of inside you — which is worse, because you lose visibility.

Monitoring and adjusting. Over time, as load and task nature evolve, initial allocations need adjustment. Monitor performance and adjust sizes as needed. Tools like Java's **VisualVM** help monitor thread usage and identify bottlenecks.

Error handling. Even with bulkheads, some requests may fail or be excessively delayed if a service goes down or becomes unresponsive. Handle these gracefully — implement a **timeout**, or use another resilience pattern like the **Circuit Breaker**.

Scalability concerns. This one deserves emphasis:

> The Bulkhead pattern improves system **resilience** but **does not directly address scalability**. If load increases significantly, simply creating more threads leads to **excessive context switching** and eventually **decreases** performance. For scale, use horizontal scaling (more machines) or asynchronous processing.

⚠️ **This is the point beginners get wrong most often.** Bulkheads bound damage; they don't add capacity. Reaching for bigger thread pools as a solution to load makes things slower, not faster, because the CPU spends its time switching between threads rather than running them.

Part 8 — Use cases

E-commerce platforms

Customer actions span searching for products, adding items to carts, and placing orders. These are interconnected but can be performed independently.

If the **search service** is overwhelmed by a traffic spike or a bug causing excessive load, without bulkheading this could slow the entire platform — **affecting even cart and ordering services**. Isolating services into separate bulkheads ensures an issue in one doesn't impact the others.

Online gaming platforms

A platform offering various games, each running as a separate service. **Player requests to one game shouldn't affect the performance of other games.** Isolating game services prevents a faulty or overloaded game from affecting the entire platform.

Financial systems

Distinct operations: account management, transaction processing, and reporting. These are interrelated but can function independently.

> **A delay in generating reports shouldn't affect the speed of transaction processing.** This is the cleanest illustration of the principle — reporting is typically heavy and slow, transactions are latency-critical, and there is no good reason for the former to interfere with the latter.

Part 9 — System design example: music streaming (Spotify-like)

A large-scale music streaming platform with services for music streaming, user management, playlist management, and recommendation generation.

Bulkheading services (coarse-grained)

User management isolated from music streaming. Even if user management becomes overloaded due to a sudden surge in new users, this doesn't affect streaming performance. **Users can still listen to music uninterrupted.**

Playlist management isolated from recommendations. Even if the recommendation service becomes slow due to the complexity of processing and analysing user data, playlist management is unaffected and users can still create and manage their playlists.

Bulkheading at a lower level (fine-grained)

Bulkheading can also be applied **within a single service**. Inside the music streaming service you could separate:

- the task of **fetching a song** (from the song database)
- the task of **streaming the song** to the user

into **separate thread pools**. So if fetching songs becomes slow due to a delay accessing the song database, this **won't slow down the streaming of songs already fetched**.

> This two-level view is the most useful takeaway from the chapter. Bulkheads apply *between* services **and** *within* a single service. The fine-grained version is often what actually saves the user experience — a user whose music keeps playing while search is broken barely notices anything is wrong.

In both cases, applying the pattern dramatically increases resilience: while one service or task is slow or failing, others continue functioning normally, providing an uninterrupted experience.

Part 10 — Bulkhead vs. Circuit Breaker

These two get confused constantly. They're complementary, not alternatives.

	Bulkhead	Circuit Breaker
Mechanism	Partitions resources	Stops calls after repeated failures
Protects against	Resource exhaustion spreading	Wasted calls to a known-dead service
When it acts	Always — it's structural	On detecting failures — it's reactive
Analogy	Watertight compartments	A fuse that trips
Limits	*How much* damage one failure can do	*How long* you keep trying

Use both. The bulkhead caps the blast radius; the breaker stops you feeding the fire. In the Order Service example, the bulkhead means Inventory can only ever consume its own 100 threads — and a breaker on Inventory means those 100 threads stop being wasted once it's clearly down.

Part 11 — Common beginner misconceptions

"Bulkhead means microservices." No. You can bulkhead inside a single monolith by giving different workloads different thread pools — and that's often the highest-value place to start.

"Bigger pools are safer." The opposite past a point. Threads cost memory and context-switching. The pool should match what the dependency can handle, not what you wish it could.

"It's the same as rate limiting." Related but different. Rate limiting caps how many requests you *accept*; bulkheading caps how many resources a category of work can *consume*.

"Isolation is a one-time build." It erodes. Teams add convenient cross-dependencies over time. It needs ongoing defence — which is why the course calls it an organisational commitment.

"Bulkheads help with load." They help with *failure*. For load you scale horizontally.

Part 12 — Check your understanding

1. Why can a slow Inventory Service cause Shipping requests to fail, when Shipping is healthy?
2. What are the two things a bulkhead isolates, and which do people forget?
3. Why does a message queue between bulkheads help contain failure?
4. Why doesn't the Bulkhead pattern help with scalability?
5. Give an example of fine-grained bulkheading inside a single service.
6. How does Bulkhead differ from Circuit Breaker?

(Answers: 1 — they share one thread pool, and Inventory's blocked threads consume all of it; 2 — code and resources, and people forget resources; 3 — the caller drops a message and moves on instead of holding a thread waiting for a struggling callee; 4 — more threads cause excessive context switching and degrade performance, so capacity comes from horizontal scaling; 5 — separate thread pools for fetching songs versus streaming already-fetched songs; 6 — bulkhead limits how much damage a failure can do and is always active, breaker limits how long you keep trying and activates on detected failures.)

Part 13 — Interview soundbite (30 seconds)

> "Bulkhead comes from ship design — watertight compartments so one hull breach doesn't sink the whole ship. In software you partition resources so a failing dependency can't consume everything. The classic case is an Order Service calling Inventory and Shipping from one shared thread pool: if Inventory gets slow, every thread ends up blocked waiting on it, so Shipping calls start failing even though Shipping is perfectly healthy. The fix is separate thread pools sized per dependency, so Inventory's problems are bounded to its own hundred threads. You can apply it between services or inside one service — for example separating song fetching from song streaming, so a slow database doesn't interrupt playback of what's already loaded. It pairs with circuit breakers: bulkhead caps how much damage a failure can do, the breaker caps how long you keep trying. The main gotchas are that sizing the pools is genuinely trial and error and needs monitoring, and that it improves resilience but not scalability — adding threads to handle more load just causes context switching, so capacity has to come from horizontal scaling."

Part 14 — Implementation

A. Runnable demo — see thread starvation, then see it fixed

This program runs the *same* workload twice: once with a shared pool, once with bulkheads. Full source: `demos/chapters/07-bulkhead/BulkheadDemo.java`

```
cd demos && ./gradlew runCh07
```

```java
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class BulkheadDemo {

    static final int POOL = 10;      // small pool so the effect is visible
    static final int REQUESTS = 20;

    // Inventory is slow (2s). Shipping is fast (50ms) and perfectly healthy.
    static String slowInventory() throws InterruptedException {
        Thread.sleep(2000);
        return "inventory-ok";
    }

    static String fastShipping() throws InterruptedException {
        Thread.sleep(50);
        return "shipping-ok";
    }

    // ---------- WITHOUT bulkheads: one shared pool ----------
    static void withoutBulkheads() throws Exception {
        ExecutorService shared = Executors.newFixedThreadPool(POOL);
        AtomicInteger shippingDone = new AtomicInteger();
        long start = System.currentTimeMillis();

        for (int i = 0; i < REQUESTS; i++) {
            shared.submit(() -> { slowInventory(); return null; });
        }
        // Shipping work arrives after the inventory flood
        for (int i = 0; i < 5; i++) {
            shared.submit(() -> {
                fastShipping();
                shippingDone.incrementAndGet();
                return null;
            });
        }

        shared.shutdown();
        boolean finished = shared.awaitTermination(3, TimeUnit.SECONDS);
        System.out.printf("  shipping completed in 3s: %d/5   (finished=%s, %dms)%n",
                shippingDone.get(), finished, System.currentTimeMillis() - start);
    }

    // ---------- WITH bulkheads: separate pools ----------
    static void withBulkheads() throws Exception {
        ExecutorService inventoryPool = Executors.newFixedThreadPool(POOL);
        ExecutorService shippingPool  = Executors.newFixedThreadPool(5);
        AtomicInteger shippingDone = new AtomicInteger();
        long start = System.currentTimeMillis();

        for (int i = 0; i < REQUESTS; i++) {
            inventoryPool.submit(() -> { slowInventory(); return null; });
        }
        for (int i = 0; i < 5; i++) {
            shippingPool.submit(() -> {
                fastShipping();
                shippingDone.incrementAndGet();
                return null;
            });
        }

        Thread.sleep(3000);
```

```
        System.out.printf("  shipping completed in 3s: %d/5   (%dms)%n",
                shippingDone.get(), System.currentTimeMillis() - start);
        inventoryPool.shutdownNow();
        shippingPool.shutdownNow();
    }

    public static void main(String[] args) throws Exception {
        System.out.println("\n=== WITHOUT bulkheads (shared pool of " + POOL + ") ===");
        withoutBulkheads();

        System.out.println("\n=== WITH bulkheads (separate pools) ===");
        withBulkheads();
    }
}
```

B. What you should see

```
=== WITHOUT bulkheads (shared pool of 10) ===
  shipping completed in 3s: 0/5   (finished=false, 3001ms)

=== WITH bulkheads (separate pools) ===
  shipping completed in 3s: 5/5   (3000ms)
```

That's the whole pattern in two lines of output. Same workload, same total resources. In the first run, twenty slow inventory tasks occupy all ten threads, and five trivial 50ms shipping tasks get **zero** of them. In the second, shipping has its own pool and finishes immediately — while inventory is just as slow as before.

Nothing was made faster. Something was made *isolated*.

C. Production — Resilience4j Bulkhead

Two flavours, and choosing correctly matters:

Type	Mechanism	Use when
`SemaphoreBulkhead`	Caps **concurrent calls** on the caller's own thread	Default choice — cheap, no extra threads
`ThreadPoolBulkhead`	Runs calls on a **separate thread pool**	You need true thread isolation, or async execution

Configuration:

```
resilience4j:
  bulkhead:                          # semaphore-based
    instances:
      inventoryService:
        maxConcurrentCalls: 25       # at most 25 in flight
        maxWaitDuration: 100ms       # then reject rather than queue forever
      shippingService:
        maxConcurrentCalls: 10
        maxWaitDuration: 100ms

  thread-pool-bulkhead:              # thread-pool-based
    instances:
      reportGeneration:
        maxThreadPoolSize: 4
        coreThreadPoolSize: 2
        queueCapacity: 20
```

Usage:

```
@Bulkhead(name = "inventoryService", fallbackMethod = "inventoryFallback")
public Inventory check(String sku) {
    return inventoryClient.get(sku);
}

private Inventory inventoryFallback(String sku, BulkheadFullException e) {
    // Bulkhead full — shed load rather than queue
    return Inventory.unknown(sku);
}
```

Kubernetes-level bulkheads — the same idea applied to whole services:

```
resources:
  requests: { cpu: "500m", memory: "512Mi" }
  limits:   { cpu: "1000m", memory: "1Gi" }   # this pod cannot starve its neighbours
```

Database connection pools are bulkheads too — and often the most valuable one:

```
spring.datasource.hikari:
  maximum-pool-size: 20
  connection-timeout: 3000     # fail fast when the pool is exhausted
```

D. How to size the pools

The most-asked question, and there's a real formula to start from — **Little's Law**:

```
concurrency = throughput × latency
```

If a dependency serves **50 requests/second** at **200ms** average latency, then `50 × 0.2 = 10` concurrent calls are in flight at steady state. Size the bulkhead a bit above that — say 15 — to absorb bursts.

The key insight: don't size it larger than the dependency can actually handle. If Inventory degrades beyond 100 concurrent requests, a bulkhead of 500 doesn't help you — it just moves the queue from your process into theirs, where you can't see it or shed it.

Then measure and adjust. `maxWaitDuration` short (~100ms) means you **shed load** rather than accumulate it, which is what you want.

E. Implementation pitfalls

1. Pools sized by guesswork. Start from Little's Law, then tune with real metrics. "200 because that's the default" is how you end up with a pool that can't protect anything.

2. Unbounded queues. `Executors.newFixedThreadPool` uses an **unbounded** `LinkedBlockingQueue`. Under sustained overload it accumulates tasks until you get an `OutOfMemoryError` — the bulkhead prevented thread exhaustion and gave you memory exhaustion instead. Use a bounded queue with an explicit rejection policy:

```
new ThreadPoolExecutor(10, 10, 0L, TimeUnit.MILLISECONDS,
    new ArrayBlockingQueue<>(50),                    // bounded
    new ThreadPoolExecutor.AbortPolicy());           // reject when full
```

3. Ignoring the `Future`. `submit()` swallows exceptions into a `Future` nobody reads — failures vanish silently. Either use `execute()` with an `UncaughtExceptionHandler`, or actually check the result.

4. Forgetting the database connection pool. Teams bulkhead their HTTP calls beautifully and leave one shared 20-connection Hikari pool, where one slow query starves every endpoint. The connection pool is usually the *scarcest* resource you own.

5. Bulkheading without timeouts. A bounded pool full of threads blocked forever is still a dead pool. Bulkhead limits *how many* can be stuck; only a timeout limits *how long*.

6. Too many small pools. Every pool holds idle threads at ~1 MB of stack each. Twenty pools of fifty threads is a thousand threads and roughly a gigabyte before you serve a request. Bulkhead by *dependency*, not by endpoint.

7. Treating it as a scaling tool. Repeated because it's the most common error: bigger pools do not mean more capacity. Past the CPU's ability to run them, more threads means more context switching and *less* throughput.

Related patterns

- **Circuit Breaker** — the natural companion; stop calling what has already failed
- **Retry** — retries inside a bounded bulkhead can't exhaust the whole system
- **Sidecar** — service meshes can enforce bulkheads at the infrastructure level
- **Event-Driven Architecture** — message queues between components are a natural isolation boundary